RELATÓRIO TÉCNICO

Trabalho Prático I - Algoritmos e Estruturas de Dados II

Árvore AVL e Árvore Rubro-Negra aplicadas ao cadastro de pacientes
Minimundo: Clínica / Atendimento

Autores

Arthur Djan
Jeziel Monteiro
Oliviê Kalil

Sumário

1. Introdução e objetivos 3

2. Minimundo, modelagem e organização do sistema 3

3. Implementação da Árvore AVL 5

4. Implementação da Árvore Rubro-Negra 7

5. Instrumentação, complexidade e testes 10

6. Metodologia experimental 12

7. Resultados observados 14

8. Análise comparativa 21

9. Limitações metodológicas 23

10. Conclusão e referências 24

1. Introdução e objetivos

Este trabalho implementa duas árvores binárias de busca balanceadas, AVL e Rubro-Negra, para armazenar a mesma coleção de pacientes. O número do prontuário é a chave única de ordenação. Nome, data de nascimento e telefone permanecem associados ao paciente e não interferem na posição ocupada nas estruturas.

Uma árvore binária de busca comum pode adquirir altura linear quando recebe chaves em ordem crescente. Nesse caso, uma busca pode percorrer quase todos os registros. As duas estruturas estudadas evitam esse comportamento mediante regras de balanceamento. A AVL controla diferenças de altura entre subárvores; a Rubro-Negra controla cores e a quantidade de nós pretos nos caminhos.

O objetivo é demonstrar o funcionamento das operações e comparar os efeitos dessas estratégias sobre altura, rotações, recolorações, caminhos de busca e tempos observados. A aplicação oferece inserção, busca, remoção, extremos, caminhamentos, exibição e validação. Os experimentos usam entradas equivalentes, sementes registradas e instâncias independentes das árvores.

As implementações foram desenvolvidas diretamente em Python, sem bibliotecas que forneçam árvores balanceadas prontas. As medições descrevem estes códigos nos cenários e no ambiente documentados; não constituem uma classificação geral de desempenho de todas as implementações AVL e Rubro-Negras.

2. Minimundo, modelagem e organização do sistema

2.1. Registro do paciente

A classe `Paciente`, definida em modelos.py, é uma dataclass imutável. A validação ocorre na criação do objeto, antes de seu cadastro.

Campo	Tipo	Regra adotada
Prontuário	Inteiro	Positivo; booleanos e outros tipos são recusados
Nome	Texto	Não pode ser vazio ou conter somente espaços
Data de nascimento	Texto	Data real no formato exato `AAAA-MM-DD`
Telefone	Texto	Obrigatório; não pode conter somente caracteres de espaçamento

O telefone conserva sua representação original, inclusive zeros iniciais e espaços de um conteúdo não vazio. Não há conversão para inteiro nem exigência adicional de máscara ou quantidade de dígitos. O modelo também verifica datas inválidas, como 29 de fevereiro em um ano não bissexto.

A imutabilidade impede alterar a chave de um paciente depois de inseri-lo. Para trocar o prontuário, é necessário remover o cadastro antigo e inserir um registro com a nova chave. As árvores armazenam o objeto recebido e preservam todos os seus atributos. Uma tentativa de inserir prontuário duplicado retorna `False` sem substituir o paciente existente.

2.2. Módulos e coordenação das árvores

Os módulos avl.py e rubro_negra.py implementam as estruturas de forma independente. Ambos aceitam objetos com atributo `prontuario`, sem importar o módulo da outra árvore. A geração dos pacientes fica em gerar_dados.py, e o contrato de integração, em integracao.py.

O menu de main.py permite escolher qual árvore será consultada ou exibida. Cadastro e remoção atualizam ambas. Um dicionário conserva o último estado confirmado para coordenação e recuperação, enquanto buscas, extremos e caminhamentos são executados nas árvores reais.

Uma atualização só é confirmada depois de conferir as duas coleções. Se houver falha parcial, a aplicação tenta reconstruir ambas a partir do estado anterior. Essa recuperação preserva os pacientes, mas pode mudar a forma das árvores e substitui as estatísticas pelas da reconstrução. Caso a recuperação também falhe, as operações ficam bloqueadas com uma mensagem explícita. O cadastro do menu permanece em memória e não persiste entre execuções.

Os experimentos são independentes do cadastro aberto no menu. O módulo experimentos.py prepara os cenários e mede as operações; coleta_experimentos.py organiza as repetições e os metadados; analise_experimentos.py produz tabelas e figuras a partir dos CSVs preservados.

Cada execução de experimentos pelo menu recebe um CSV exclusivo na pasta `resultados/menu/`. O destino utiliza um identificador UUID, preservando o histórico e os arquivos de execuções anteriores. A medição utiliza novas instâncias e mantém os pacientes, a forma e as estatísticas do cadastro aberto.

2.3. Contrato comum

As interfaces equivalentes permitem aplicar as mesmas sequências às duas estruturas. A altura conta níveis de nós reais: árvore vazia tem altura 0 e uma folha tem altura 1. Filhos ausentes e sentinelas não entram na quantidade de pacientes nem no contador de visitas.

Operação	Retorno ou efeito
`inserir(paciente)`	`True` ao inserir; `False` para chave inválida ou duplicada
`buscar(prontuario)`	Paciente armazenado ou `None`
`remover(prontuario)`	`True` ao remover; `False` para chave inválida ou inexistente
`minimo()` e `maximo()`	Paciente de menor ou maior chave; `None` na árvore vazia
Três caminhamentos	Novas listas de pacientes em pré-ordem, em ordem ou pós-ordem
`altura()` e `quantidade()`	Altura em níveis e quantidade de pacientes
`exibir()`	Texto com a estrutura, sem impressão automática
`validar()`	Booleano que informa a validade da estrutura

As interfaces também oferecem consulta e reinício de estatísticas e controle do modo detalhado. As operações inválidas e as buscas ou remoções ausentes não encerram o programa. Não há conversão silenciosa de textos em chaves inteiras.

3. Implementação da Árvore AVL

3.1. Nós, alturas e fator de balanceamento

Cada nó da AVL contém o paciente, uma cópia do prontuário, as referências aos filhos esquerdo e direito e a altura da subárvore. A árvore vazia e os filhos ausentes são representados por `None`. Não há referência ao pai: as operações recursivas retornam a nova raiz de cada ramo para reconectar as subárvores.

As definições utilizadas são:

```
h(None) = 0
h(no) = 1 + maximo(h(esquerda), h(direita))
FB(no) = h(esquerda) - h(direita)
```

Um fator positivo indica maior altura à esquerda; um fator negativo indica maior altura à direita. A propriedade AVL exige que o fator de todos os nós permaneça entre `-1` e `+1`. Além disso, as alturas armazenadas devem corresponder às alturas reais e a ordenação binária deve ser estrita.

O menor número de nós de uma AVL de altura `h` satisfaz `N(h) = 1 + N(h - 1) + N(h - 2)`, com `N(0) = 0` e `N(1) = 1`. O crescimento dessa recorrência é do tipo Fibonacci, limitando a altura a uma função logarítmica da quantidade de pacientes.

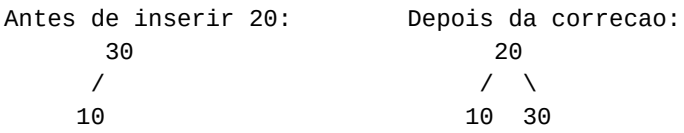
3.2. Inserção e rotações

A inserção percorre recursivamente o ramo determinado pela chave e cria um nó com altura 1 ao encontrar um filho ausente. Durante o retorno da recursão, `_reequilibrar()` atualiza a altura e verifica o fator de cada ancestral. Se a chave já existe, a operação termina sem alterar o cadastro ou os contadores.

Caso	Condição	Correção
LL	Fator maior que 1; filho esquerdo com fator não negativo	Rotação à direita
RR	Fator menor que -1; filho direito com fator não positivo	Rotação à esquerda
LR	Fator maior que 1; filho esquerdo com fator negativo	Esquerda no filho e direita no ancestral
RL	Fator menor que -1; filho direito com fator positivo	Direita no filho e esquerda no ancestral

As rotações reorganizam referências e preservam a ordem relativa das chaves. A altura do nó rebaixado é atualizada antes da altura do promovido, pois a segunda depende da primeira. Uma correção dupla é contabilizada como duas rotações simples. O contador de pacientes aumenta uma única vez por inserção.

Como exemplo, preparar a árvore com `30, 10` e inserir `20` produz o caso LR:



O prontuário 20 é inicialmente filho direito de 10. O fator de 30 chega a +2, enquanto o de 10 é -1. A primeira rotação, à esquerda no 10, transforma o triângulo em linha; a segunda, à direita no 30, promove 20 à raiz. A altura permanece 2, a quantidade passa de dois para três e a operação soma duas rotações de inserção. Os contadores são reiniciados depois da preparação para isolar a operação apresentada.

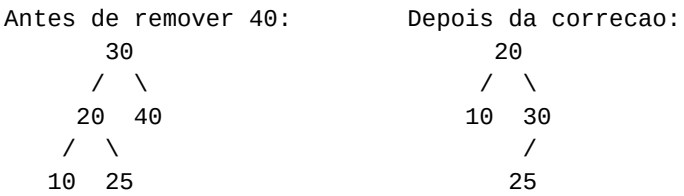
3.3. Busca, caminhamentos e remoção

A busca é iterativa. Compara o prontuário com a chave do nó atual e escolhe o próximo ramo até encontrar o paciente ou alcançar `None`. O mínimo segue os filhos esquerdos, e o máximo, os direitos. Os caminhamentos usam as ordens raiz-esquerda-direita, esquerda-raiz-direita e esquerda-direita-raiz. Nenhuma dessas consultas altera alturas ou realiza rotações.

Na remoção, uma folha é retirada retornando `None`; um nó com um filho é substituído por esse filho. Com dois filhos, o registro completo do sucessor em ordem substitui o registro do alvo, e a ocorrência antiga do sucessor é removida na subárvore direita. A identidade do paciente sobrevivente e seus atributos são preservados. A quantidade diminui uma única vez.

O retorno recursivo atualiza alturas e reequilibra os ancestrais. A redução de altura pode exigir correções em vários níveis, por isso a remoção continua até a raiz. Quando o filho mais alto do nó desequilibrado possui fator zero, a correção é simples.

O exemplo seguinte prepara `30, 20, 40, 10, 25` e remove a folha 40:



A exclusão deixa 30 com fator +2 e seu filho 20 com fator zero. Uma rotação à direita em 30 promove 20; o antigo filho direito 25 de 20 passa para a esquerda de 30. A altura total continua 3, a quantidade cai de cinco para quatro e há uma rotação de remoção. Na estrutura final, as buscas por 20, 25 e 99 visitam, respectivamente, um, três e dois nós reais.

3.4. Validação da AVL

`validar()` utiliza uma pilha explícita em pós-ordem. Transporta limites de ordenação herdados dos ancestrais e recalcula as alturas depois de verificar os filhos. Assim, a decisão sobre o equilíbrio não depende apenas dos valores armazenados nos próprios nós.

A rotina verifica prontuários válidos, correspondência entre chave e paciente, ordenação estrita, alturas armazenadas inteiras e corretas, fator entre -1 e +1, quantidade alcançada

e ausência de ciclos ou nós compartilhados. A pilha permite rejeitar estruturas corrompidas profundas sem depender do limite de recursão. A validação não corrige a árvore nem modifica suas métricas.

4. Implementação da Árvore Rubro-Negra

4.1. Nós, cores e folhas NIL

Cada nó real contém paciente, cópia do prontuário, cor e referências aos filhos e ao pai. As cores são `PRETO` e `VERMELHO`. Existe um sentinela preto `_nil` por instância, compartilhado pelos filhos ausentes daquela árvore e usado também como raiz da árvore vazia. O sentinela não guarda paciente, seus filhos apontam para ele próprio e ele é o pai da raiz real.

Durante uma remoção, o pai de `_nil` pode apontar temporariamente para a região que perdeu um nó preto. Isso permite aplicar o mesmo algoritmo a um substituto real ou ausente. Ao terminar, seu pai volta a apontar para o próprio sentinela. Sua cor permanece preta. Árvores distintas não compartilham nós ou sentinelas.

A estrutura preserva as seguintes propriedades:

1. Todo nó é preto ou vermelho, e a raiz real é preta.
2. Todas as folhas ausentes, representadas por NIL, são pretas.
3. Um nó vermelho não possui filho vermelho.
4. Todos os caminhos de um mesmo nó até NIL possuem igual quantidade de pretos.
5. A ordenação binária é estrita, sem chaves duplicadas.

Na contagem usada aqui, um caminho inclui o nó inicial, se preto, e o NIL final. A igualdade das contagens de pretos e a ausência de vermelhos consecutivos limitam a altura a `h <= 2 * log2(n + 1)`. Essa altura se refere aos níveis de nós reais e não deve ser confundida com a contagem de pretos.

Nos diagramas compactos desta seção, `P` significa preto e `V`, vermelho. Todas as folhas NIL pretas são omitidas apenas para simplificar o desenho; continuam presentes na representação interna e nas verificações de altura negra.

4.2. Inserção, recolorações e rotações

A inserção localiza a posição pela chave e conecta um novo nó vermelho com dois filhos NIL. Criá-lo vermelho preserva a quantidade de pretos nos caminhos. O conflito que pode surgir é um pai também vermelho. `_corrigir_insercao()` examina pai, avô e tio para resolver esse conflito.

Para o pai situado à esquerda do avô, os casos são:

Caso	Configuração	Correção
1	Pai e tio vermelhos	Pai e tio ficam pretos; avô fica vermelho; análise continua no avô
2	Tio preto e triângulo LR	Rotação à esquerda no pai prepara uma linha
3	Tio preto e linha LL	Pai fica preto, avô vermelho e ocorre rotação à direita no avô

Quando o pai está à direita, os casos são espelhados. O triângulo RL é transformado com uma rotação à direita no pai, e a linha RR é corrigida com uma rotação à esquerda no avô. Ao terminar, a raiz é tornada preta.

Preparar `30, 10` e inserir `20` exige a correção dupla:

```
Antes de inserir 20:        Depois da correcao:
       30P                         20P
      /                           /   \
    10V                         10V   30V
```

A rotação à esquerda em 10 transforma o triângulo LR em linha. O prontuário 20 muda de vermelho para preto, 30 muda de preto para vermelho e a rotação à direita em 30 promove 20. A altura permanece 2. Isolada da preparação, essa inserção registra duas rotações e duas recolorações.

O caso de tio vermelho pode ser observado preparando `10, 5, 15` e inserindo `1`. Os nós 5 e 15 ficam pretos, o avô 10 fica vermelho e, por ser a raiz, volta a preto ao final. A operação registra quatro mudanças efetivas de cor e nenhuma rotação. A altura passa de 2 para 3. A quantidade de pretos aumenta igualmente em todos os caminhos.

4.3. Remoção e correção da altura negra

A remoção começa pela localização do prontuário. Um alvo sem filho esquerdo real é substituído pelo direito; sem filho direito real, pelo esquerdo. Com dois filhos, o sucessor em ordem ocupa sua posição. Nesse caso, o nó sucessor inteiro é movido, preservando seu paciente. Se ele não for filho direto do alvo, seu filho direito assume primeiro sua posição antiga.

A cor original do nó retirado de sua posição determina a necessidade de correção. Com dois filhos, essa é a cor do sucessor antes de ele herdar a cor do alvo. Portanto, a remoção de um alvo preto nem sempre executa o laço de correção: um sucessor originalmente vermelho pode ocupar sua posição e ficar preto sem deixar déficit na posição de onde saiu.

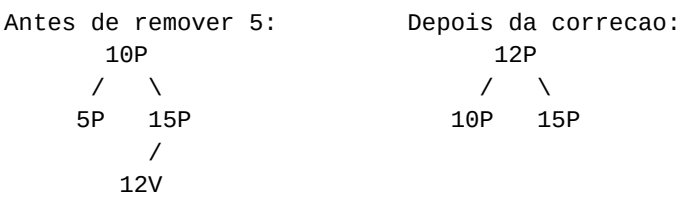
Se a exclusão retirar um preto de um caminho, o algoritmo associa um "preto extra" conceitual ao substituto. Nenhuma terceira cor é armazenada. Se o substituto for vermelho, torná-lo preto resolve o déficit. Se ele for preto e ainda não for a raiz, são analisados irmão e sobrinhos.

Para o substituto à esquerda do pai, o irmão fica à direita; o sobrinho próximo é o filho esquerdo do irmão, e o distante é seu filho direito.

Caso	Configuração	Tratamento
1	Irmão vermelho	Recolore irmão e pai; rotaciona o pai à esquerda para obter um irmão preto
2	Irmão preto com filhos pretos	Torna o irmão vermelho e leva o déficit para o pai
3	Próximo vermelho e distante preto	Recolore e rotaciona o irmão à direita, preparando o caso 4
4	Irmão preto com distante vermelho	Irmão herda a cor do pai; pai e distante ficam pretos; rotação à esquerda elimina o déficit

O caso 2 pode propagar a correção por vários ancestrais. Os casos 1 e 3 preparam configurações posteriores, e o caso 4 encerra a correção. Quando o substituto está à direita, os lados são trocados: os casos 1 e 4 rotacionam à direita, e o caso 3, à esquerda.

Preparar `10, 5, 15, 12` e remover a folha preta 5 exige os casos 3 e 4:



O substituto é NIL. O irmão 15 é preto, o sobrinho próximo 12 é vermelho e o distante é NIL preto. O caso 3 torna 12 preto e 15 vermelho, rotacionando o irmão à direita. O caso 4 torna 15 preto e rotaciona 10 à esquerda. A altura cai de 3 para 2, e são registradas duas rotações e três recolorações.

Outro exemplo prepara `10, 5, 15` e remove a raiz 10. O sucessor vermelho 15 é transplantado e recebe a cor preta. O resultado tem raiz 15 preta e filho esquerdo 5 vermelho. Há uma recoloração e nenhuma rotação; a altura continua

2. O objeto do paciente 15 mantém sua identidade.

4.4. Validação Rubro-Negra

`validar()` usa uma pilha explícita para transportar os limites de ordenação, o pai esperado e a quantidade de pretos acumulada. Ao alcançar NIL, compara a contagem com a dos outros caminhos. Também verifica raiz preta, cores válidas e ausência de vermelhos consecutivos.

A verificação inclui os vínculos do sentinela, pais dos nós, correspondência entre a chave armazenada e a do paciente, quantidade alcançada e ausência de ciclos ou nós compartilhados. Os limites herdados permitem detectar violações em relação a ancestrais distantes. A validação não altera pacientes, cores, vínculos ou estatísticas.

5. Instrumentação, complexidade e testes

5.1. Contadores e modo detalhado

As duas árvores retornam uma cópia de um dicionário com cinco campos. As recolorações da AVL são sempre zero, mantendo a mesma interface de coleta.

Campo	Significado
`rotacoes_insercao`	Rotações simples acumuladas nas inserções
`rotacoes_remocao`	Rotações simples acumuladas nas remoções
`recoloracoes_insercao`	Mudanças efetivas de cor nas inserções
`recoloracoes_remocao`	Mudanças efetivas de cor nas remoções
`nos_visitados_ultima_busca`	Nós reais percorridos pela última busca

Uma correção dupla conta duas rotações. Recolorações contam cada mudança efetiva, inclusive a promoção da raiz para preto e a herança de cor do sucessor na remoção. Criar um nó vermelho não conta; atribuir novamente a mesma cor também não. Se um nó mudar de cor e depois retornar à original, as duas mudanças são contabilizadas. NIL nunca é recolorido.

A busca conta o nó encontrado, quando houver, e exclui os filhos ausentes. Uma busca inválida ou em árvore vazia registra zero. Somente `buscar()` atualiza essa medida, substituindo o valor anterior. `reiniciar_estatisticas()` zera os cinco campos sem apagar pacientes nem alterar o modo detalhado.

Com o modo detalhado ativo, as atualizações mostram a chave envolvida, a região afetada, o caso de correção, as rotações e a estrutura final. A Rubro-Negra mostra também as recolorações. O método `exibir()` apenas devolve o texto; a impressão depende de chamada explícita ou do modo detalhado. Esse modo fica desativado durante as medições de tempo.

5.2. Complexidade

Os custos abaixo pressupõem estruturas válidas e modo detalhado desligado.

Operação	AVL	Rubro-Negra
Inserção, busca e remoção	O(log n)	O(log n)
Mínimo e máximo	O(log n)	O(log n)
Três caminhamentos	O(n)	O(n)
Altura	O(1), mantida na raiz	O(n), calculada pelos nós
Quantidade	O(1)	O(1)
Validação	O(n)	O(n)
Consulta e reinício de estatísticas	O(1)	O(1)

Cada rotação altera uma quantidade constante de referências. Na AVL, as atualizações percorrem a cadeia de ancestrais pelo retorno recursivo; na Rubro-Negra, as correções acompanham os pais. Em árvores válidas, a profundidade é logarítmica. Os caminhamentos produzem listas com n pacientes; os validadores usam O(n) de memória auxiliar para identificar nós e conferir propriedades.

A exibição percorre a estrutura e gera texto cujo tamanho depende da indentação. Quando esse texto é produzido e impresso após uma atualização detalhada, seu custo soma-se ao da operação. Esse custo de diagnóstico não deve ser confundido com o custo das atualizações utilizadas nos experimentos.

5.3. Testes automatizados

A suíte específica da AVL contém 32 testes e a da Rubro-Negra, 26. Ambas inspecionam diretamente os nós, além de chamar `validar()`, evitando que um defeito no validador oculte um erro de balanceamento. Cobrem árvores vazias, duplicatas, entradas inválidas, caminhos, identidade dos pacientes, mudanças de raiz, sucessores, contadores e corrupção de referências ou propriedades.

Os testes da AVL incluem os quatro casos de rotação na inserção e na remoção, correções com filho de fator zero e desequilíbrio em mais de um nível. Há 3.600 operações mistas, distribuídas entre três sementes, comparadas com um dicionário independente. O conjunto maior insere 10.000 chaves crescentes e remove todas em ordem aleatória.

Na Rubro-Negra, os testes cobrem tio vermelho, os casos de remoção e seus espelhos, exclusão de folhas pretas, transplantes e restauração do sentinela. As operações mistas totalizam 3.000 chamadas com sementes explícitas e um dicionário de referência. Outro cenário insere e depois remove 10.000 pacientes. Nos conjuntos maiores, as invariantes são verificadas a cada 1.000 alterações, evitando custo quadrático de verificação.

Os testes de integração usam as classes reais e o modelo `Paciente`. Conferem coleções equivalentes nas duas árvores, coordenação do cadastro, métricas e exportação CSV. As visitas de busca são comparadas com uma contagem independente dos caminhos nos nós. Há também testes do modelo, do gerador, das falhas parciais, da coleta repetida e da análise dos resultados.

As regressões do menu verificam execuções consecutivas, preservação do histórico e tratamento de falhas de exportação. A conferência das métricas também é exercitada com valores negativos, booleanos, tipos incorretos, números não finitos e alturas incompatíveis com as quantidades. A verificação integrada executou 134 testes com sucesso, incluindo os gráficos com a dependência opcional instalada.

Na raiz do repositório, a suíte é executada com:

```
python -m unittest discover -s tests -v
```

Em ambientes cujo executável é `python3`, basta substituir o comando. A aprovação dos testes fornece evidência para os cenários exercitados, sem provar ausência de defeitos em todas as sequências possíveis. Os exemplos de árvores deste relatório foram conferidos com as saídas das implementações.

6. Metodologia experimental

6.1. Dados e cenários

Os pacientes são sintéticos. Os nomes recebem a indicação "Fictício", as datas ficam entre 1940-01-01 e 2020-12-31, e os telefones começam com `00`. Cada conjunto usa prontuários consecutivos únicos de 1 até seu tamanho. O gerador utiliza uma instância local de `random.Random(seed)`, preservando o estado aleatório global.

Foram avaliados 100, 1.000 e 10.000 pacientes com semente inteira 42. O código prepara uma sequência crescente e uma cópia embaralhada. Em cada cenário, as duas árvores recebem os mesmos objetos e a mesma ordem de operações.

Cenário	Preparação	Operações medidas
A: inserção crescente	Instâncias vazias	Inserir todos os prontuários crescentes
B: inserção aleatória	Novas instâncias vazias	Inserir o mesmo conjunto embaralhado
C: busca	Construir com a sequência de B	Consultar existentes e inexistentes em lotes separados
D: remoção	Reconstruir o conjunto completo para cada percentual	Remover 10%, 30% e 50%

Em C, um lote consulta todas as chaves existentes, na ordem embaralhada; outro consulta igual quantidade de chaves ausentes, de `maior_prontuario + 1` até `maior_prontuario + quantidade`. As árvores podem ser reutilizadas entre esses dois lotes, porque as buscas preservam a coleção e a forma da estrutura.

Em D, o subconjunto removido é um prefixo da sequência embaralhada, de comprimento `quantidade * percentual // 100`. Os percentuais não acumulam exclusões: cada cenário começa com árvores recém-construídas. Com 10.000 pacientes, são removidos 1.000, 3.000 e 5.000 registros, restando 9.000, 7.000 e 5.000, respectivamente.

6.2. Medição e conferência

`time.perf_counter()` mede o tempo total do lote, incluindo o laço Python e a construção da lista de retornos. O CSV conserva segundos; tabelas e figuras apresentam milissegundos. Não se trata de tempo individual por operação.

Geração dos dados, embaralhamento, construção prévia para C/D, leitura das alturas, reinício e leitura das estatísticas, validação, conferência de retornos, impressão e gravação ficam fora do intervalo cronometrado. O tempo inclui a instrumentação executada internamente pelas próprias operações. Nenhum lote cronometrado imprime a árvore.

A altura inicial é lida imediatamente antes do cronômetro. Em A/B ela é zero; em C/D corresponde à árvore previamente construída. A altura final é obtida depois das operações. Na coleta de visitas de C, uma primeira passagem mede as buscas; uma segunda, fora do cronômetro, repete as mesmas chaves e lê a quantidade de nós visitados imediatamente após cada chamada. A média divide a soma das visitas pelo número de buscas do lote.

Os contadores são reiniciados antes de cada lote quando o método está disponível. Diferenças entre leituras anteriores e posteriores isolam as operações mesmo quando não há reinício. A/B utilizam os contadores de inserção, e D utiliza os de remoção. Depois de cada lote, são conferidos retornos, quantidade, todos os campos dos pacientes em ordem e invariantes estruturais.

Valores não aplicáveis são exportados vazios. Em C, rotações e recolorações não são métricas coletadas; em A/B/D, a média de visitas não se aplica. Zeros disponíveis são preservados, inclusive as recolorações da AVL. A análise recusa lotes incompletos, hashes divergentes ou métricas estruturais inconsistentes.

A leitura dos resultados confere ainda os tipos e os limites dos campos. Alturas, quantidades, operações e contadores presentes devem ser inteiros não negativos, sem booleanos; tempos e médias presentes devem ser finitos e não negativos. As alturas precisam corresponder ao estado vazio ou não vazio e às quantidades do cenário. Ausências legítimas e zeros são preservados.

6.3. Repetições, ambiente e rastreabilidade

A coleta definitiva realizou cinco execuções sequenciais dos três tamanhos. As instâncias são novas em cada execução, mas o processo Python é o mesmo. Cada tamanho produz 14 linhas: duas de A, duas de B, quatro de C e seis de D. São 42 linhas por execução e 210 no lote completo. A ordem entre estruturas é fixa: AVL antes da Rubro-Negra em cada cenário.

Informação	Valor registrado
Sistema operacional	Windows 11, versão 10.0.26200
Python	3.15.0, interpretador de 64 bits
Arquitetura	AMD64 / 64 bits
Processador informado	Intel64 Family 6 Model 183 Stepping 1, GenuineIntel
Cronômetro	QueryPerformanceCounter(), monotônico
Resolução informada	0,0000001 s; não equivale à precisão garantida
Coleta	24/10/2026, 14:50:03 a 14:50:08, America/Manaus, UTC-04:00
Parâmetros	Semente 42; tamanhos 100, 1.000 e 10.000; cinco repetições
Commit medido	138a4bfc89fcace472dc6f31dd4f10403524a67b

O nome comercial do processador não foi identificado pelo procedimento usado. A versão completa do Python e os horários por execução estão no manifesto do lote. Esse arquivo registra comando, estado Git e hashes SHA-256 dos fontes e dos CSVs. Os bytes originais, inclusive as terminações de linha geradas no Windows, foram conservados para permitir a conferência dos hashes.

O lote pequeno de validação usou dez pacientes e uma execução. Ele não participa da estatística definitiva. O CSV histórico também foi preservado e não recebeu alturas iniciais

retroativas. Somente os cinco CSVs listados no manifesto do lote definitivo compõem os resultados apresentados aqui.

Para cada combinação de tamanho, cenário e árvore, a análise calcula mediana, mínimo, máximo e amplitude absoluta dos cinco tempos. Não foram descartadas observações extremas. Métricas estruturais precisam coincidir entre repetições; divergências interrompem a análise, em vez de serem ocultadas por médias.

7. Resultados observados

7.1. Integridade da coleta e alturas

As cinco execuções concluíram as 210 linhas previstas, todas com `valido=True` e sem limitações de instrumentação. Os 42 grupos apresentaram métricas estruturais idênticas entre repetições. Os 60 campos vazios de rotações, os 60 de recolorações e os 150 de visitas correspondem aos casos não aplicáveis, sem indicar perda de medições.

As alturas finais dos dois cenários de inserção foram:

Pacientes	A: AVL	A: Rubro-Negra	B: AVL	B: Rubro-Negra
100	7	11	8	8
1.000	10	17	12	12
10.000	14	24	16	16

Na inserção crescente, a AVL ficou mais baixa nos três tamanhos. Na inserção aleatória, as alturas foram iguais para a semente utilizada. Em buscas, as alturas permaneceram iguais às de B. Nas remoções, a altura da AVL foi menor ou igual à da Rubro-Negra nos grupos observados. A Figura 1 reúne as alturas finais de todos os cenários.

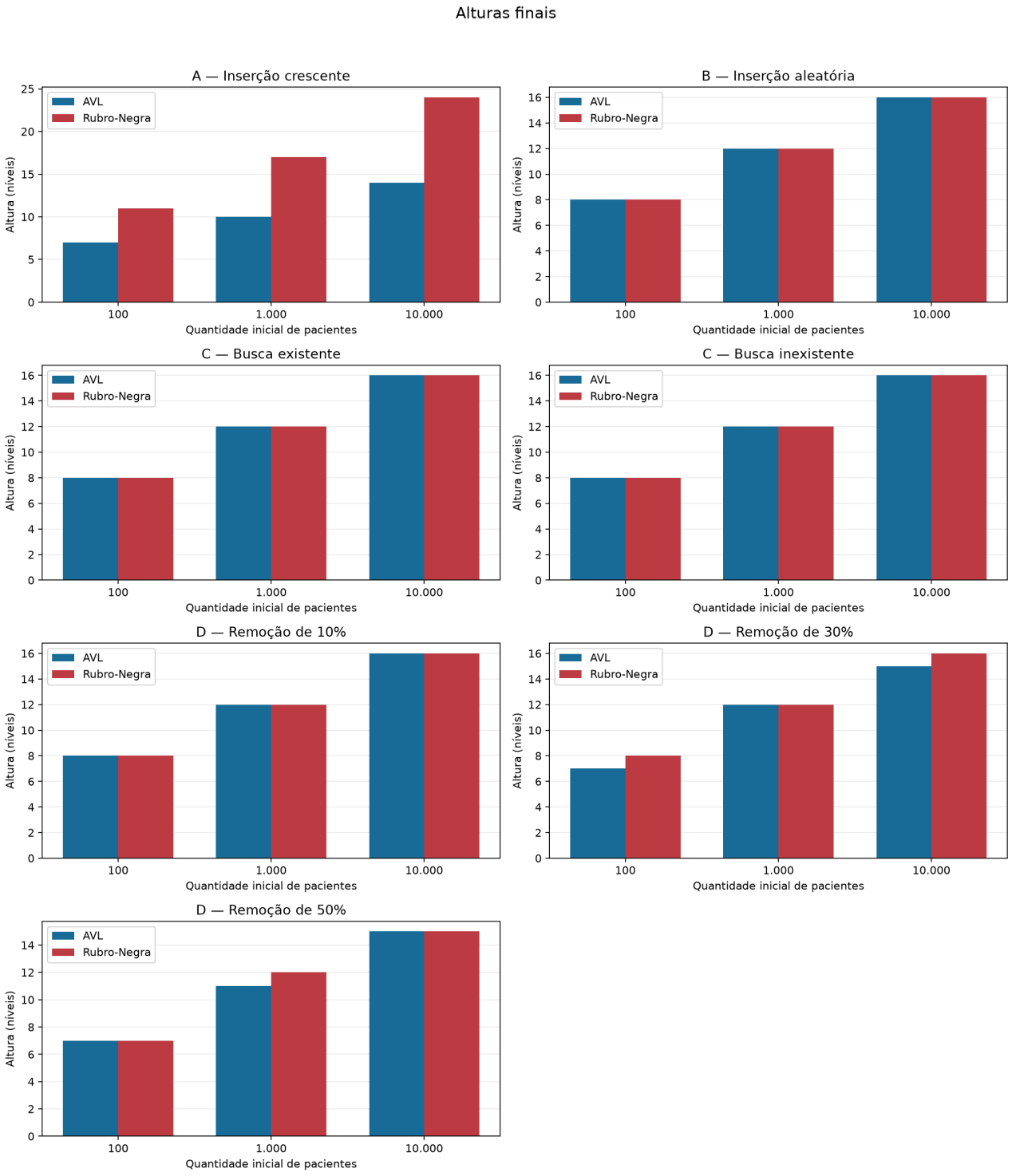


Figura 1: Alturas finais das duas árvores em todos os cenários.

7.2. Rotações e recolorações

O recorte de 10.000 pacientes mostra como os eventos de balanceamento variam com a operação. Nas remoções, cada percentual parte de 10.000 registros.

Cenário	Rotações AVL	Rotações Rubro-Negra	Recolorações Rubro-Negra
A: crescente	9.986	9.976	49.877
B: aleatória	6.946	5.806	23.230
D: 10%	476	405	1.786
D: 30%	1.337	1.273	5.782
D: 50%	2.143	2.187	10.079

A AVL não utiliza cores e, portanto, registra zero recolorações em todos esses cenários. A contagem de rotações simples é comparável entre as estruturas; recolorações são um tipo adicional de trabalho da Rubro-Negra, não uma equivalência direta com rotações. A Figura 2 apresenta as rotações dos três tamanhos, separadas por cenário.

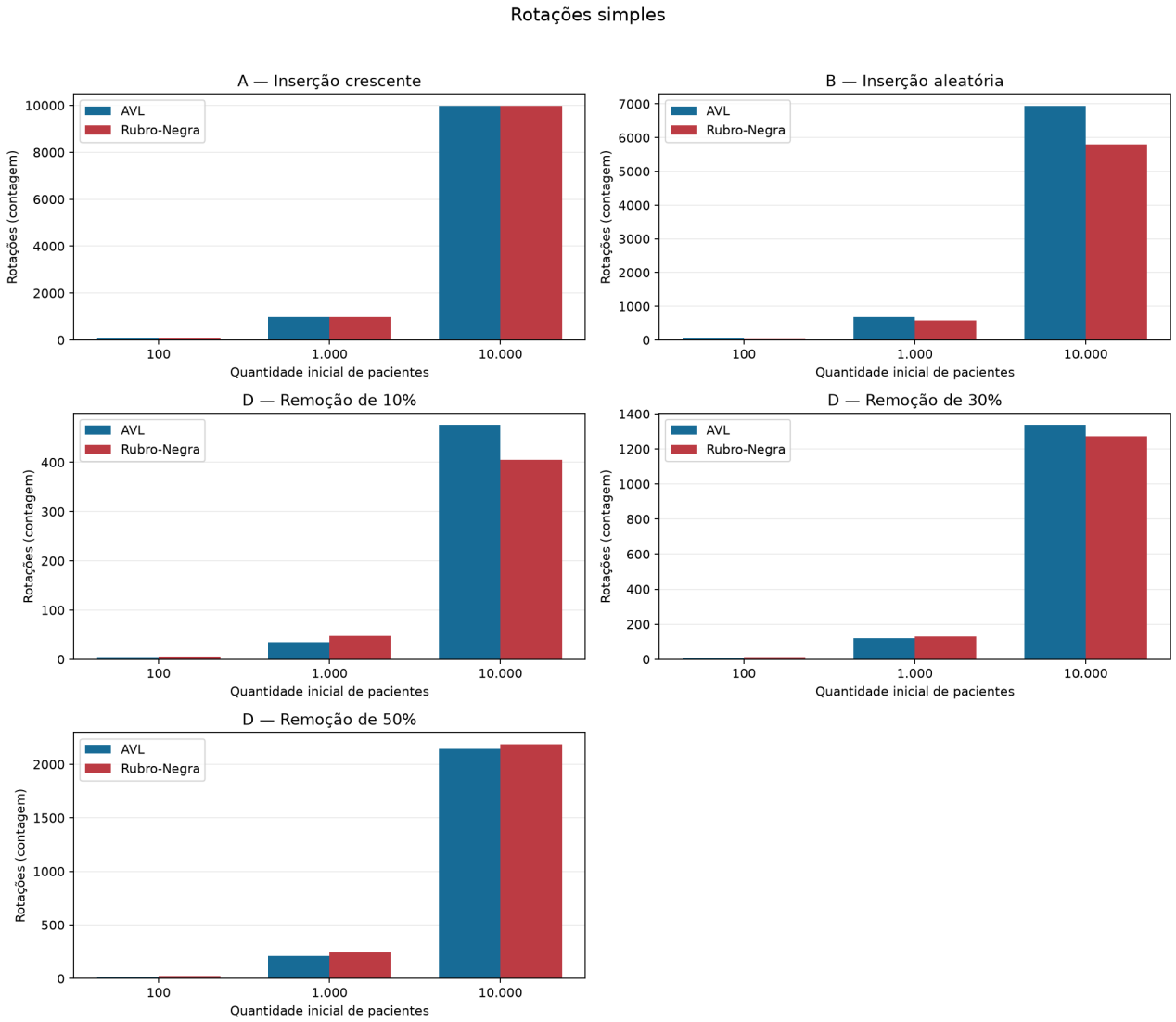


Figura 2: Rotações simples acumuladas nas inserções e remoções.

A inserção crescente de 10.000 pacientes exigiu 49.877 mudanças de cor na Rubro-Negra; a aleatória, 23.230. Nas remoções, o total cresceu de 1.786 para 10.079 entre os lotes de 10% e 50%. Os números descrevem lotes de quantidades diferentes e não devem ser interpretados como custo constante de cada remoção. A Figura 3 mostra as recolorações sem atribuir eventos de cor à AVL.

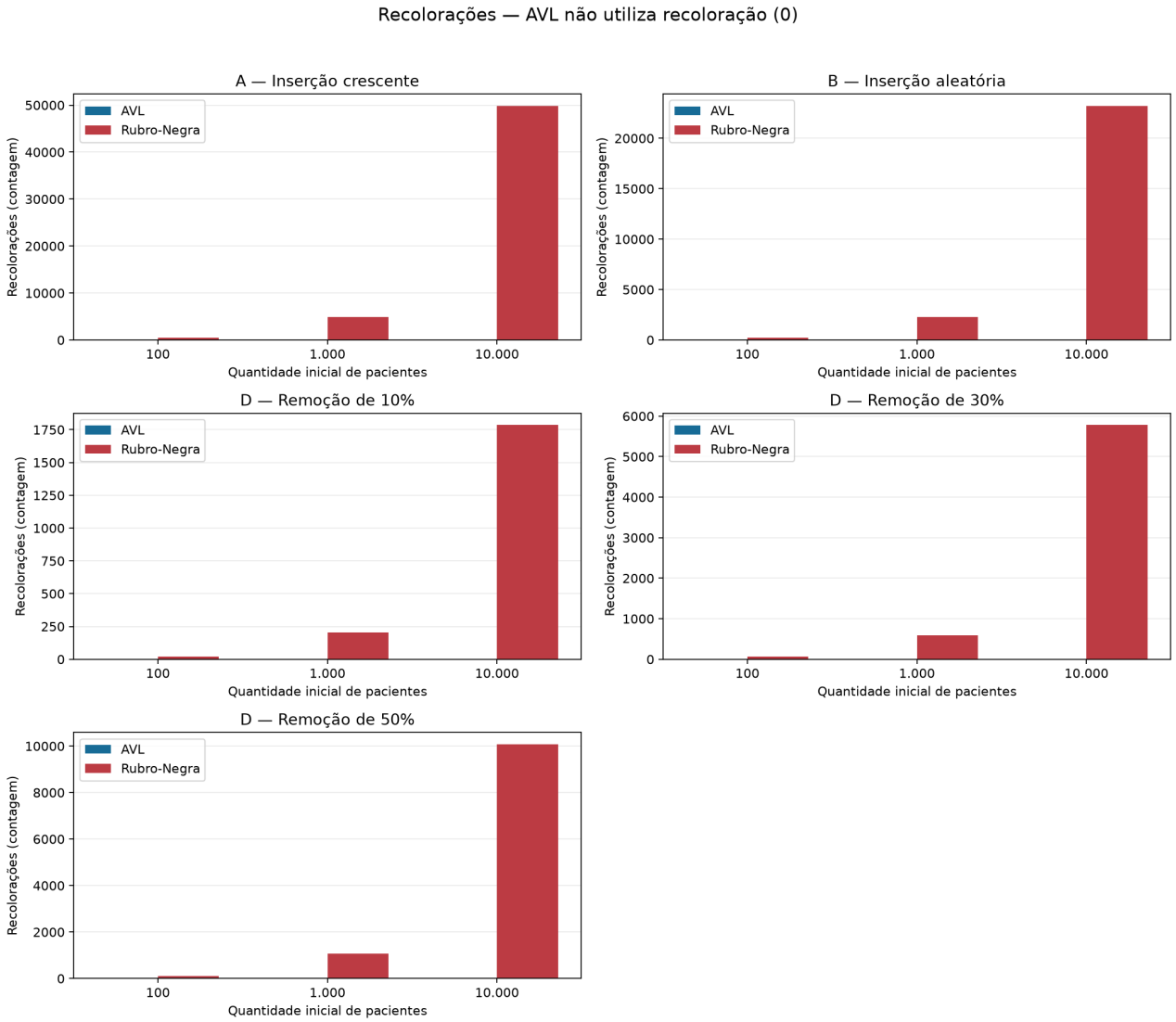


Figura 3: Recolorações; os valores da AVL são zero porque a estrutura não usa cores.

7.3. Caminhos das buscas

As médias de nós reais visitados, separando buscas bem-sucedidas e ausentes, foram as seguintes. Cada lote contém tantas consultas quanto o tamanho do conjunto, sem contar a passagem suplementar de coleta no tempo medido.

Pacientes	Existentes: AVL	Existentes: Rubro-Negra	Ausentes: AVL	Ausentes: Rubro-Negra
100	5,8400	5,9400	7	6
1.000	9,1790	9,2650	10	10
10.000	12,5611	12,6458	14	13

A AVL visitou ligeiramente menos nós nas buscas existentes dos três tamanhos. Nas ausentes, a Rubro-Negra visitou menos nós com 100 e 10.000 pacientes e igual quantidade com 1.000. Como todas as chaves inexistentes ficam acima do máximo, esses

últimos valores caracterizam o caminho à direita das árvores, e não uma distribuição ampla de consultas malsucedidas. A Figura 4 mantém os dois tipos de busca separados.

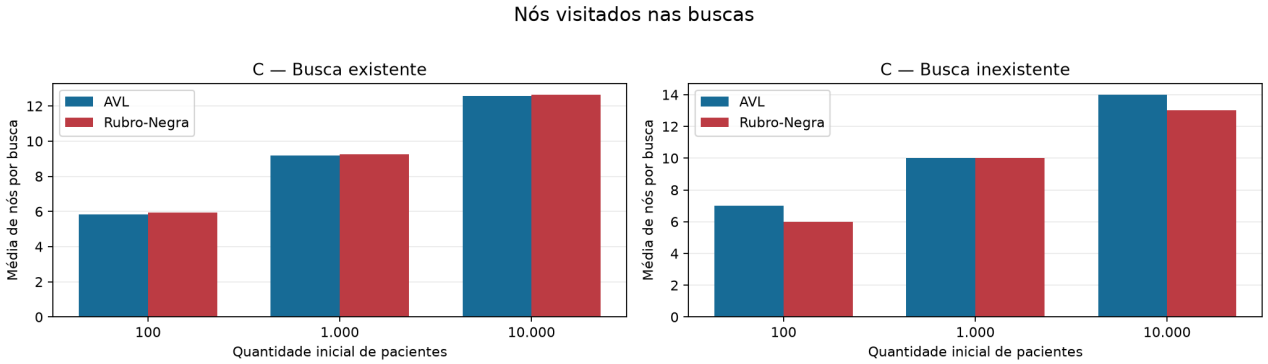


Figura 4: Média de nós visitados por busca, distinguindo chaves existentes e inexistentes.

7.4. Tempos dos lotes

A tabela apresenta as medianas dos cinco tempos para 10.000 pacientes, em milissegundos. A quantidade de operações varia conforme o percentual removido.

Cenário	Operações no lote	Mediana AVL (ms)	Mediana Rubro-Negra (ms)
A: inserção crescente	10.000	42,1328	26,3644
B: inserção aleatória	10.000	39,2417	18,6951
C: busca existente	10.000	5,3287	5,3193
C: busca inexistente	10.000	4,6109	4,1872
D: remoção de 10%	1.000	3,8688	1,7211
D: remoção de 30%	3.000	10,8166	4,9784
D: remoção de 50%	5.000	17,6388	8,5509

Nesse recorte, a Rubro-Negra apresentou menores medianas em todos os cenários. As buscas existentes tiveram valores muito próximos e faixas observadas sobrepostas. Com 100 pacientes, a AVL teve mediana de busca existente menor: 0,0293 ms contra 0,0310 ms. Portanto, nem mesmo esta coleta apresenta a mesma ordenação de tempos em todos os tamanhos e operações.

As figuras de tempo mostram mediana e extremos. Cada painel informa unidade e escala própria, com eixo vertical partindo de zero. A Figura 5 separa as ordens de inserção crescente e aleatória.

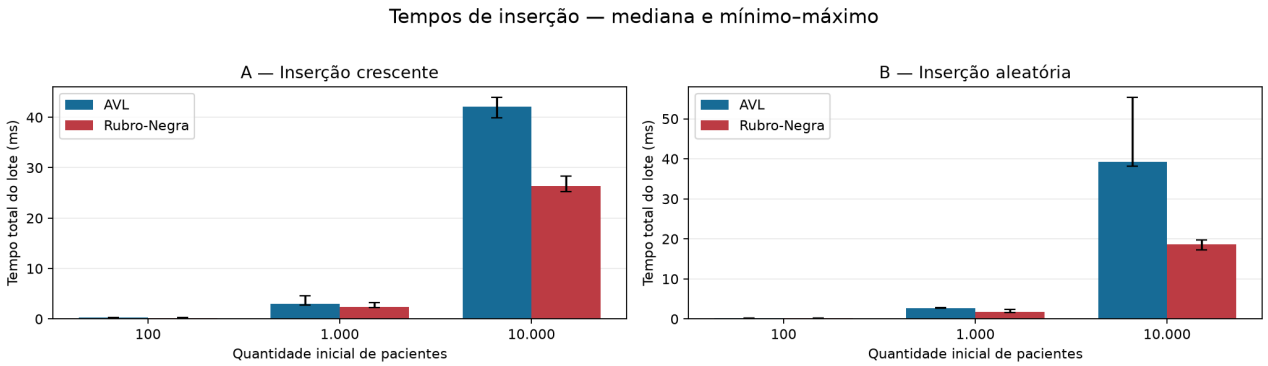


Figura 5: Tempos totais de inserção, com medianas e extremos das cinco execuções.

A variabilidade precisa acompanhar a leitura das medianas. Alguns exemplos do recorte de 10.000 pacientes são:

Cenário e árvore	Mínimo (ms)	Máximo (ms)	Amplitude (ms)
B: AVL	38,1260	55,2967	17,1707
C existente: AVL	5,0689	6,4716	1,4027
C existente: Rubro-Negra	5,0920	12,8242	7,7322
D 50%: AVL	17,2224	27,7859	10,5635
D 50%: Rubro-Negra	8,0548	9,2206	1,1658

As observações extremas foram preservadas. Não foi identificada uma causa específica para elas. A Figura 6 distingue buscas existentes, ausentes e os percentuais de remoção. As hastes representam mínimo e máximo observados, não intervalos de confiança.

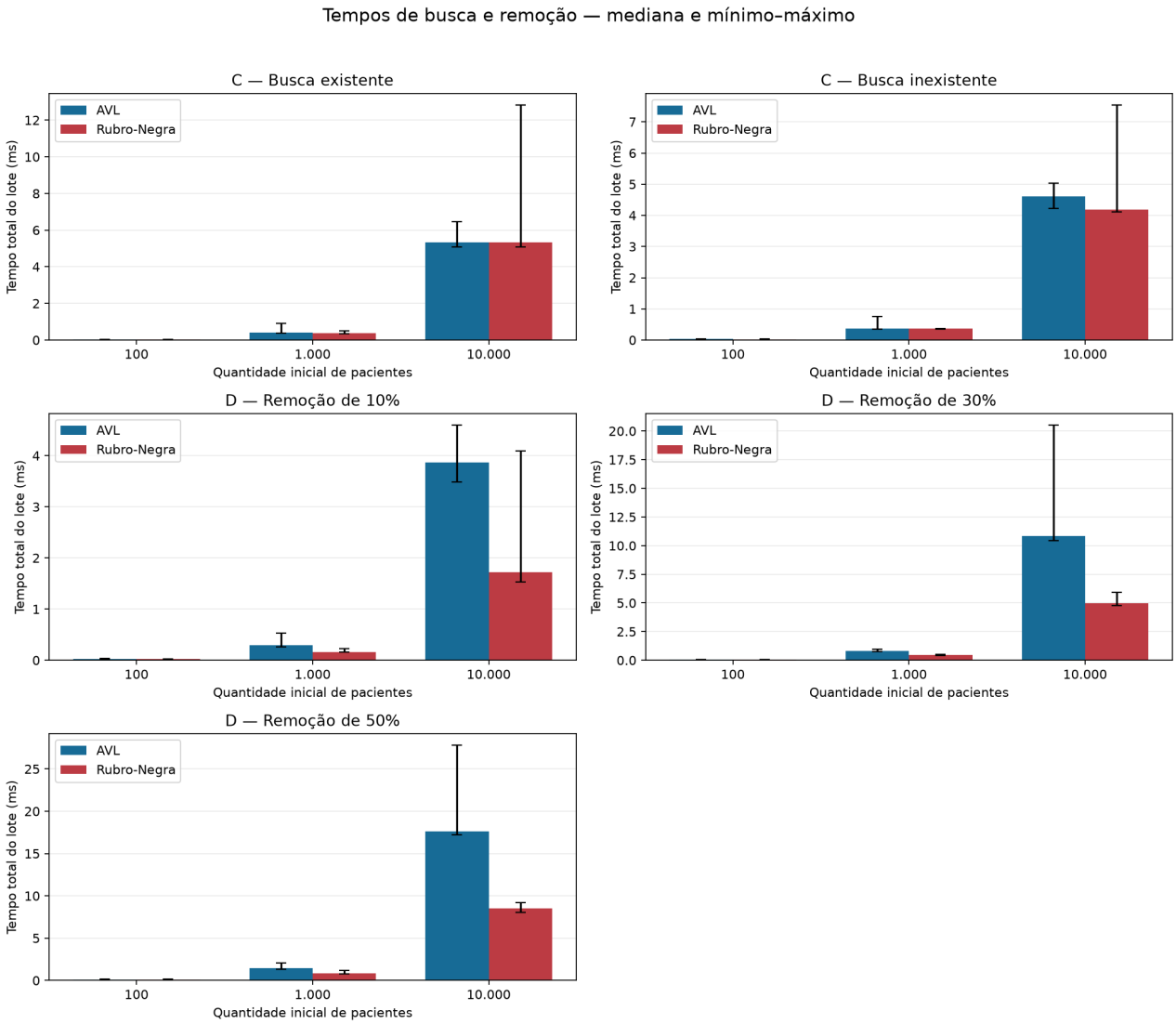


Figura 6: Tempos totais de busca e remoção, com medianas e extremos das cinco execuções.

Não se devem comparar alturas visuais de barras entre painéis com limites verticais diferentes. Entradas menores podem gerar barras baixas na escala linear; os valores exatos dos três tamanhos permanecem nas tabelas de tempos e no resumo CSV.

8. Análise comparativa

8.1. Estrutura de menor altura

A AVL teve menor altura nos três cenários de inserção crescente: 7 contra 11, 10 contra 17 e 14 contra 24. Na inserção aleatória, ambas terminaram com alturas 8, 12 e 16. As buscas conservaram essas formas. Nas remoções observadas, a AVL ficou com altura menor ou igual à Rubro-Negra, dependendo do tamanho e percentual.

Com 10.000 pacientes e remoção de 30%, por exemplo, a AVL passou de 16 para 15, enquanto a Rubro-Negra permaneceu em 16. Com remoção de 50%, ambas passaram de 16 para 15. Esses resultados são coerentes com estratégias diferentes: a AVL impõe um limite local mais estrito às alturas; a Rubro-Negra permite caminhos de comprimentos diferentes desde que preserve suas regras de cor.

8.2. Quantidade de rotações

Nas inserções A e B dos três tamanhos, a AVL realizou mais rotações simples. Com 10.000 chaves crescentes, a diferença foi pequena: 9.986 contra 9.976. Na ordem aleatória, os totais foram 6.946 e 5.806. Portanto, a ordem de entrada afetou o trabalho de ambas, mas não na mesma proporção.

Nas remoções não houve uma ordem única válida para todos os casos. Com 100 e 1.000 pacientes, a Rubro-Negra realizou mais rotações nos três percentuais. Com 10.000, a AVL realizou mais nos lotes de 10% e 30%; no de 50%, a Rubro-Negra realizou 2.187 contra 2.143. A comparação precisa considerar operação, tamanho e sequência, em vez de atribuir um único total típico a cada estrutura.

8.3. Papel das recolorações

As recolorações aparecem como outro mecanismo de correção da Rubro-Negra. Alguns conflitos são resolvidos apenas por mudanças de cor; outros combinam essas mudanças com rotações. O exemplo de tio vermelho demonstra uma inserção com quatro recolorações e nenhuma rotação.

Na coleta de 10.000 pacientes, a ordem crescente exigiu mais do que o dobro das recolorações da ordem aleatória: 49.877 contra 23.230. As remoções também produziram milhares de mudanças de cor. Esses eventos representam trabalho efetivo, mas não têm o mesmo custo de uma rotação nem podem ser somados a ela como se todos os eventos fossem unidades equivalentes de tempo.

8.4. Relação entre altura e tempo

Menor altura não garantiu menor tempo. A inserção crescente de 10.000 pacientes deixou a AVL com altura 14 e a Rubro-Negra com 24; ainda assim, as medianas foram 42,1328 ms e 26,3644 ms, respectivamente. As estruturas também podem ter a mesma altura e caminhos médios diferentes, como as buscas sobre as árvores aleatórias de altura 16.

O tempo depende do conjunto de ações executadas: comparações, manutenção de alturas ou cores, chamadas, recursão ou laços, referências e instrumentação. Esses fatores são uma interpretação possível das diferenças, não uma medição isolada de suas causas. A coleta não permite atribuir o resultado a uma única linha do código ou a uma característica de hardware específica.

8.5. Efeito da ordem de inserção

A ordem crescente produziu árvores mais altas na Rubro-Negra do que a ordem aleatória, para os três tamanhos. Na AVL ocorreu o inverso nesta semente: a sequência crescente terminou com alturas 7, 10 e 14, e a aleatória, com 8, 12 e 16. A regra de balanceamento limita a altura, mas não impõe uma forma única para um mesmo conjunto de chaves.

Para 10.000 registros, a ordem aleatória reduziu as rotações e a mediana de inserção nas duas estruturas em relação à crescente. Na Rubro-Negra, também reduziu as recolorações. Esse resultado se refere a uma única sequência aleatória por tamanho. Outras sementes podem produzir formas e contagens diferentes, mesmo preservando as propriedades de balanceamento.

8.6. Diferenças entre as operações

As diferenças de tempo mais evidentes do recorte de 10.000 registros ocorreram nas atualizações. A mediana da inserção aleatória foi 39,2417 ms na AVL e 18,6951 ms na Rubro-Negra; na remoção de 50%, foi 17,6388 ms contra 8,5509 ms. Nas buscas existentes, as medianas ficaram próximas de 5,32 ms em ambas.

Busca não executa balanceamento e seu custo depende do caminho seguido. Nas atualizações, além da localização, ocorrem manutenção das propriedades e eventos de correção. A leitura conjunta de caminhos, alturas, contadores e tempos evidencia essas diferenças. Como cada lote pode ter quantidade distinta de operações, o tempo total de remover 10% não deve ser comparado ao de inserir 100% como se fossem cargas iguais.

9. Limitações metodológicas

9.1. Entradas e amostragem

Todos os prontuários ausentes são maiores do que o maior cadastrado. Suas buscas seguem o caminho à direita, sem representar chaves abaixo do mínimo ou em lacunas internas. A metodologia controla esse padrão igualmente nas duas estruturas, mas limita a generalização das médias e dos tempos de buscas malsucedidas.

Uma única semente fornece apenas uma ordem aleatória por tamanho. As cinco repetições mantêm as mesmas entradas e caracterizam uma variabilidade limitada do tempo, sem ampliar a variedade de formas construídas. Não foram calculados intervalos de confiança nem realizados testes de significância. A sobreposição dos extremos de busca e a pequena diferença entre medianas não fundamentam uma afirmação de diferença estatisticamente significativa.

9.2. Ambiente e execução

Os tempos dependem do interpretador, sistema operacional, processador, frequência efetiva e possíveis atividades concorrentes. Não houve isolamento de núcleo, controle de frequência nem monitoramento da carga externa. A descrição do processador disponível não identificou seu nome comercial. A resolução do cronômetro não elimina variabilidade nem assegura precisão equivalente.

A ordem AVL/Rubro-Negra foi fixa. Não houve etapa dedicada de aquecimento nem reinício do processo entre repetições. Caches, coleta de lixo e operações anteriores, inclusive validações fora do intervalo cronometrado, podem influenciar as chamadas posteriores. Esses efeitos não foram medidos separadamente.

O custo do laço e da lista de retornos não foi subtraído. A instrumentação das árvores permaneceu ativa e entrou no tempo. Assim, os números caracterizam o código instrumentado utilizado, e não uma implementação abstrata sem contadores. O custo O(n) de consultar a altura na Rubro-Negra ficou fora do cronômetro, assim como a leitura O(1) da altura na AVL.

9.3. Integridade, ferramentas e alcance das conclusões

Os resultados históricos, a coleta pequena e a tentativa incompleta de análise não foram misturados ao lote definitivo. A tentativa no sandbox gerou tabelas, mas falhou ao carregar integralmente a dependência gráfica; seus arquivos e o erro foram preservados em uma pasta separada. A análise completa usou os mesmos CSVs, sem refazer tempos ou modificar medições.

Sistema, coleta e tabelas utilizam a biblioteca padrão. A geração das seis figuras usa opcionalmente `matplotlib`, versão 3.11.2 no procedimento registrado. As figuras são derivadas das tabelas, e não fontes independentes de medições. Os hashes permitem verificar arquivos e associar os resultados ao código registrado; por si só, não demonstram a qualidade estatística do experimento.

As invariantes e os retornos foram conferidos após os lotes. Essa verificação detecta inconsistências exercitadas, mas não substitui a análise de todos os casos possíveis. Os resultados empíricos também não provam complexidade assintótica. As conclusões permanecem restritas aos códigos, às entradas, aos tamanhos e ao ambiente descritos.

10. Conclusão e referências

10.1. Conclusão

As duas implementações mantiveram a mesma coleção de pacientes e ofereceram operações equivalentes. A validação das propriedades, os testes dirigidos e os cenários reproduzíveis forneceram evidências para inserção, busca, remoção, caminhamentos e instrumentação das estruturas.

A AVL apresentou menor altura na inserção crescente; na ordem aleatória usada, as alturas foram iguais. Ela realizou mais rotações nas inserções, enquanto nas remoções a comparação variou conforme o tamanho e o percentual. A Rubro-Negra combinou rotações com recolorações e, no recorte de 10.000 pacientes, apresentou menores medianas de tempo. As buscas existentes tiveram tempos próximos e não sustentam uma afirmação de diferença estatística significativa.

A relação entre altura, correções e tempo precisa ser interpretada em conjunto. Uma árvore mais baixa não foi necessariamente mais rápida. A ordem de entrada alterou as formas e os contadores mesmo com o mesmo conjunto de chaves. A comparação evidencia os efeitos dos dois mecanismos de balanceamento sem atribuir superioridade universal a uma das estruturas.

10.2. Fontes técnicas e artefatos

As explicações e os exemplos detalhados das implementações estão nas seções de AVL e Rubro-Negra. A metodologia detalhada conserva a descrição do procedimento e o recorte numérico usados na consolidação deste relatório.

Referências técnicas para as propriedades e operações:

- MIT OpenCourseWare: Red-black Trees, Rotations, Insertions, Deletions.
- Dartmouth: Balanced Binary Search Trees, 2-3-4 and Red-Black Trees.
- Código da AVL, código da Rubro-Negra e suas documentações de AVL e Rubro-Negra.

Artefatos que sustentam os resultados:

- Manifesto e metadados do lote definitivo, com referências aos cinco CSVs originais.
- Resumo completo em CSV, com estatísticas de tempo e métricas estruturais.
- Tabelas completas de tempos e métricas estruturais.
- Verificação de integridade e consistência, incluindo hashes e identificação das figuras.
- Suíte da AVL, suíte da Rubro-Negra e testes de integração real.